

第一阶段个人工作小结

1、本阶段工作内容

前面几周我主要负责前期需求设计文档的编写，以及后端底层的架构搭建和部分核心逻辑开发。文档方面，我和队友一起完成了《需求规格说明书》和《系统设计说明书》，我个人主要负责梳理总体架构图，以及算法逻辑模块的撰写。代码开发方面，进度比原计划稍微快一点。我先写好了项目全局的 `docker-compose.yml` 和各个微服务的 `Dockerfile`，确保 Java 业务层、Python 算法层和 PostgreSQL 数据库的环境彻底隔离开。接着完成了 Python 算法端的工程初始化，配置 `requirements.txt`，并封装了对接 DeepSeek 大模型的客户端逻辑（`deepseek_client.py`）。然后为了赶进度，我提前把 Python 连数据库的底层逻辑写完了（包括 `db.py` 和 `client_db.py`），测试了对 `pgvector` 向量数据的增删改查。最后，我还搭了一个基础的 API 路由（`chat_routes.py`），把大模型调用的接口暴露出来，方便后续联调。

2、遇到的问题以及解决思路

遇到的第一个问题主要是异构微服务之间的通信选型争议。因为我们是 Java 加 Python 的双后端架构，一开始我想直接用 gRPC 来做内部通信，觉得性能更好。但实际测试发现，写 `proto` 文件和反复编译太耗时间了，严重拖慢了前期的开发节奏。

组内讨论后，决定先退一步，改用轻量级的 HTTP/REST 方案。通过 Docker 的内部网络来保证通信安全，只要对齐 JSON 数据格式就能快速开发，后续如果真有并发瓶颈再考虑升级。

然后还有一个问题是 Python 容器打包体积过大导致启动慢，刚开始写 Python 端 `Dockerfile` 的时候，直接用了默认的 Python 镜像，加上安装了一堆算法库，打出来的镜像快好几个 G。每次修改代码后重新 `build` 和启动都很费时间。最后我把基础镜像换成了更轻量的 `python:3.11-slim` 版本。然后重新梳理了 `requirements.txt`，清掉了一些没用上的包，并在 `pip install` 语句后面加上了 `--no-cache-dir` 指令，把缓存清理掉，最终把镜像体积缩减到了可以接受的范围。

然后还遇到 `pgvector` 插件的数据类型不匹配报错的问题，我在写 `client_db.py` 测试向量数据入库时，一直报错。发现是因为 Python 原生的 `List` 类型和 PostgreSQL 里 `pgvector` 插件定义的 `vector` 类型不能直接对应，导致 ORM 框架在序列化时崩溃。然后我查了下 `pgvector` 的官方文档，发现需要引入一个专门的适配库，或者手动做类型转换。为了求稳，我写了个工具函数，在执行 SQL 插入前，强制把 Python 数组转化成数据库能识别的特殊字符串格式（像 `[1.1, 2.2, ...]` 这样，特定的文本表示），才终于算把向量存取跑通。

3、下一阶段计划

接下来的阶段重点是死磕 RAG（检索增强生成）的具体逻辑。我计划先把教案切片和文本向量化的核心代码写完。等这块弄好后，就开始跟谢巧对接，把整个后端的链路联调起来，尽快把从 Java 网关接收请求，到传给 Python 处理，再到 DeepSeek 生成答案，到最后返回给前端的完整数据流能跑通。